Amazon Leadership Principles

Master Stories in STAR Format

Roshan Jha · Software Development Engineer interview preparation

How to use this document

Each master story is one real project told in full. For any question, tell the same events but **lead with the part the principle cares about** — the mapping table under each story shows which questions it answers and what to emphasize.

Colour key: text in [orange brackets] needs your real number or detail. **[VERIFY]** marks a detail that was inferred or invented to make the story complete — keep it only if it is true, and replace it otherwise.

Stories 8–10 are **templates**: the technical parts come from your CV, but the people parts were invented. Replace them with real moments before your interview.

Use any one story in **at most two interviews**. Amazon interviewers compare notes, and the same example across a whole loop looks like a thin record.

Coverage plan

Story	Primary principles
1. StonksAI filings	Are Right, A Lot
2. UPI Rewards WebView	Bias for Action
3. Deployment freeze	Dive Deep, Learn and Be Curious
4. Referral product	Ownership
5. MQTT migration	Invent and Simplify, Frugality
6. Experimentation platform	Think Big, Success and Scale Bring Broad Responsibility
7. Accessibility fix	Insist on the Highest Standards, Customer Obsession, Deliver Results
8. Stale cache (template)	Earn Trust
9. Unified entitlements (template)	Have Backbone; Disagree and Commit
10. On-call improvements (template)	Strive to Be Earth’s Best Employer (+ Hire and Develop the Best, non-manager)

Master Story 1

StonksAI Real-Time Filings Pipeline

StonksAI (Co-Founder) · May 2025 – Present

Primary principle: Are Right, A Lot

Situation

In May 2025 I co-founded StonksAI, a product for Indian retail investors. As a small, bootstrapped team we had no users, no usage data and no clear idea which problem to solve first. We sent a survey to retail investor groups and got about [X] responses, but the feedback was too general to build on, things like “too much noise” or “I hear about news too late.”

Task

Decide what to build, and then make the system behind it work, with very little data, no benchmarks and a tight budget.

Action — creating our own evidence

Instead of relying on the survey alone, I looked for something measurable. I took about [N] corporate filings on NSE and BSE (quarterly results, order wins) and compared when each PDF was posted with how the stock price moved. In many cases prices moved within [X minutes] of the filing appearing. That turned “news reaches me late” into a concrete gap: by the time most retail investors saw a filing, the price had often already moved. We decided to build a product that gets filings to users as fast as possible. **[VERIFY]** Most respondents hadn’t asked for filings specifically; they described the symptom, and the data showed the cause.

Action — choosing how to build it

[VERIFY] I weighed three ways to get filings quickly:

- Buy a third-party data feed: fast to launch, but expensive, and our main differentiator would depend on a vendor.
- Poll the exchanges aggressively: fastest, but high risk of being throttled or blocked, and wasteful.
- Build our own crawler with smart polling: more work up front, but we control speed and cost.

We chose to build our own.

Action — the polling decision

The key open question was how often to check the exchanges. Too slow and we lose our value; too fast and we overload public exchange sites, risk being blocked and waste money. **[VERIFY]** My first instinct was very aggressive polling, but my co-founder pushed back on cost and blocking risk, and a quick test showed [throttling responses]. So we:

- started with a moderate [X-second] interval during market hours and slower polling after hours;
- made the interval configurable at runtime, so it could change without a redeploy;
- deduplicated filings by ID and added exponential backoff on errors or throttling;
- logged two timestamps per filing — when the exchange posted it and when we detected it — creating a “detection latency” metric that didn’t exist before.

Action — tuning with real data

After [2–3 weeks], the data showed filings clustered around [market open / close] and during results season. We moved to an adaptive schedule: faster in those peak windows, slower in quiet periods.

Result

Median detection latency reached about [X seconds], with [zero / minimal] blocking from the exchanges and infrastructure cost of about Rs [X] per month, far below a paid data feed. This pipeline became the foundation for StonksAI's filings fetching, caching and streaming services and fed our real-time content feeds (Home, Pulse, Catalyst).

Reflection

When data is thin, I look for a signal I can measure myself, make the first decision easy to reverse, and add measurement so the system produces the data I was missing. **[VERIFY]** If I did it again, I'd log detection latency from the very first prototype instead of adding it after launch.

How to angle this story

Principle	Questions it answers	What to emphasize
Are Right, A Lot	Q1–Q8	All eight answers prepared from this story (see angles below).
Bias for Action	Q5 move forward vs gather info; Q1 calculated risk	Launched with a reasonable, reversible interval and let real data guide tuning.
Customer Obsession	Q3 anticipated an unstated need	Users said "news is late"; your price analysis found the real need.
Dive Deep	Q6 created a metric; Q3 in-depth analysis	Detection latency is a metric you created, and it drove the adaptive schedule.
Think Big	Q3 novel approach	Targeted the gap between a filing and the price move.
Invent and Simplify	Q5 usual approach wouldn't work	Fixed polling was too slow or too risky, so you designed adaptive polling with backoff.
Frugality	Q4 quality vs cost; Q3 limited resources	Own crawler instead of a paid feed: near-feed speed at a fraction of the cost.
Ownership	Q4 short-term vs long-term	A paid feed was the quick win; building your own kept the core advantage under your control.
Success and Scale	Q3 impact on a third party; Q1 impact beyond customer	Adaptive polling and backoff kept you a "polite" client of public exchange infrastructure.
Learn and Be Curious	Q5 new area of an existing space	Public filings, seen through a new lens: timing vs price movement.

Question-by-question angles

Question	How to tell it
Q1 Not enough data	Survey too thin → own price analysis → reversible starting interval → instrumented and tuned.
Q2 Strategic decision without benchmarks	Lead with the alternatives: paid feed vs aggressive vs conservative vs adaptive polling, and how you reduced risk.
Q3 Many sources of input	Investors (survey), market data, co-founder (cost), exchange constraints (rate limits) — and how you weighed which failure hurt most.
Q4 Bad decision	**[VERIFY]** Shipped the first crawler without detection-latency logging, so you tuned blind for [X weeks]. Lesson: measure before you optimize.
Q5 Error in judgment	**[VERIFY]** Assumed filings arrived evenly during trading hours; data showed clustering at open/close and results season. Fixed with adaptive polling.
Q6 Your idea wasn't best	**[VERIFY]** Your aggressive-polling idea; co-founder flagged cost and blocking; a test confirmed it; adaptive polling was better.

Question	How to tell it
Q7 Brought perspectives together	**[VERIFY]** Active traders (speed), long-term investors (understanding), co-founder (cost), you (engineering) — one pipeline serving separate feeds.
Q8 Sought diverse perspectives (DEI)	**[VERIFY]** Surveyed beyond your network (first-time, non-metro, older, non-English-first investors); learned raw PDFs alone weren’t enough for newer investors.

Follow-up questions to prepare for

- How did you know the price moved because of the filing, not other news?
- Is scraping the exchanges allowed? How did you stay within their limits?
- Why not buy a data feed?
- Was the starting interval correct? What would you do differently?

Master Story 2

UPI Rewards WebView Performance

Slice (Software Developer) · June 2022 – Jan 2024

Primary principle: Bias for Action

Situation

When I joined Slice, I worked on Android development. I was given the Android side of UPI Rewards: after completing a UPI payment, a user could play a game and win a reward. It was one of the company's top priorities for user acquisition, and marketing had a campaign planned around it. I built all the screens and integrated the games through a WebView, and the feature was functionally complete.

Task

While testing on a real device, I saw each game took about 20 seconds to open. Nobody had flagged it. **[VERIFY]** The games came from [another team / an external vendor], so load time wasn't strictly my responsibility, and the feature technically "worked." But a user who had just paid would be staring at a loading screen for 20 seconds, and many would leave before seeing the reward.

Action — digging into the cause

[VERIFY] Before fixing anything, I profiled the load. Most of the 20 seconds came from downloading the game's assets (JavaScript, images, sounds) from scratch on every open, plus WebView start-up time. Nothing was reused between sessions, and our app had never used WebView caching anywhere, so there was no existing pattern to copy.

Action — deciding to act

A proper caching solution would take about two working days. Releases went out every Monday, so doing it during the week meant UPI Rewards would miss the release or ship slow. **[VERIFY]** I told my [lead / manager] about the problem, the options and my plan: work through the weekend, keep the change contained, and have it ready for review before the cutoff. **[VERIFY]** I chose caching over pre-loading the WebView (risky for memory on low-end phones) or asking the games team to shrink their bundle (unavailable over the weekend), because caching was something I could fully build and test myself.

Action — building it carefully

[VERIFY] I learned how WebView caching works on Android quickly, then built it with safeguards:

- caching only for the game WebViews, so the rest of the app was unaffected;
- a fallback to normal network loading if the cache failed;
- versioned assets, so updated games wouldn't be stuck on old cached files;
- testing on [X] devices, including low-end phones and slow networks.

Result

The change went out in Monday's release. Game load time dropped from about 20 seconds to about [X seconds] on repeat opens. UPI Rewards launched on schedule for the marketing campaign and contributed to [user acquisition / engagement numbers]. **[VERIFY]** The caching approach was later [reused for other WebView screens / documented for the team].

Reflection

I found the problem late because I only tested performance once the feature was complete. Now I test on real devices early, so problems like this get fixed during normal working time without a weekend push.

How to angle this story

Principle	Questions it answers	What to emphasize
Bias for Action	Q1 calculated risk; Q2 tight deadline; Q6 proactive fix	The quick decision, the weekend deadline, and how you limited the downside.
Customer Obsession	Q5 evaluated customer experience; Q2 above and beyond	You tested as a real user right after a payment and refused a 20-second wait.
Ownership	Q1 outside your responsibility	The games weren’t yours and nobody asked you to fix it.
Insist on the Highest Standards	Q1 refused to compromise; Q4 standards vs delivery	Wouldn’t ship it slow, and didn’t sacrifice the release date either.
Deliver Results	Q1 tight deadline; Q2 unanticipated obstacle	The obstacle appeared late; you still shipped Monday.
Dive Deep	Q2 root cause	Profiling where the 20 seconds went before choosing a fix.
Learn and Be Curious	Q2 outside comfort area; Q1 deeper expertise	New to Android and WebView caching; learned it fast enough to ship safely.
Frugality	Q5 no extra resources	Solved on-device, with no new infrastructure or other teams.
Earn Trust	Q6 uncovered a significant problem	Raising the issue and plan with your lead before acting, open about the risk.

Question-by-question angles

Question	How to tell it
Bias for Action Q1 — calculated risk	Three options (ship slow, delay, weekend fix); chose speed for a high-value launch; contained the risk with scope limits, fallback, versioning and device testing.
Bias for Action Q2 — tight deadline	About two days; filter was “which good-enough option can I own and verify in time?”; wrote down skipped alternatives for follow-up.

Follow-up questions to prepare for

- What exactly did you cache, and how? (Android WebView already has a basic HTTP cache — be precise about what yours added.)
- Why not just delay a week?
- What if the cache had broken something?
- If you’d had a week, what would you have done?

Master Story 3

The 15-Second Deployment Freeze

[VERIFY] [Slice / StonksAI]

Primary principle: Dive Deep · Learn and Be Curious

Situation

Our backend services consumed events from Kafka (**[VERIFY]** [payment events, reward triggers, notifications]). Every rolling deployment caused all message processing to freeze for about 15 seconds — not just the topic whose consumers were restarting, but every topic, on every pod. At peak traffic that meant thousands of events queuing with nothing processed. Then, after about 15 seconds, everything resumed as if nothing had happened. There was no message loss.

Task

Because nothing was lost, it was easy to dismiss as a harmless blip. But we deployed [X times a week], and each freeze delayed user-facing actions like **[VERIFY]** [reward credits or notifications] and created a backlog burst that loaded downstream services. **[VERIFY]** I took on finding out why a single pod restart could stop processing across every topic.

Action — confirming the pattern

[VERIFY] On our Grafana dashboards I lined up throughput and consumer lag against deployment times. Processing dropped to zero at every deploy, across all topics, and lag spiked by [X] messages. Consumer logs showed that at every pod restart, each consumer revoked all its partitions and rejoined the group.

Action — root cause

[VERIFY] Kafka spreads a topic's partitions across the consumers in a group, and rebalances whenever a member leaves or joins. Three things combined to make our rebalances freeze everything:

- One shared consumer group across all topics: a restart affecting one topic triggered a rebalance for every topic.
- Eager ("stop-the-world") rebalancing: with the default assignor every consumer gives up all partitions, even ones that don't move, and nothing is processed until reassignment finishes.
- No graceful shutdown: pods were killed without closing the consumer, so the group only learned a member had left when its session timeout expired — matching the ~15 seconds. The new pod joining then triggered a second rebalance.

Action — the fix

[VERIFY]

- Graceful shutdown: a Kubernetes pre-stop hook and longer termination grace period, so the consumer closed cleanly and left the group immediately.
- Cooperative rebalancing: switched to the cooperative sticky assignor, so only partitions that actually move are paused.
- Separate consumer groups per topic [or domain], so a restart in one area couldn't pause unrelated topics.
- (Optional) Static membership — a stable consumer ID per pod — so a quick restart didn't trigger a rebalance at all.

I rolled these out to [one service first], verified the dashboards during a test deployment, then applied them to [the other X services].

Result

[VERIFY] The freeze dropped from about 15 seconds across all topics to [under 1 second / no visible pause], limited to the partitions being moved. Peak lag during deploys fell from [thousands] to [X] messages. Deployments became a non-event and **[VERIFY]** the team became comfortable deploying during peak hours. **[VERIFY]** I documented the fix as [a shared consumer config / runbook] so new services got it by default.

Reflection

“No data loss” isn’t the same as “no impact.” And a perfectly consistent symptom — exactly 15 seconds, every time — almost always points to a configuration value worth investigating.

How to angle this story

Principle	Questions it answers	What to emphasize
Dive Deep	Q2 root cause; Q1 complex problem; Q3 in-depth analysis	Matching dashboards to deploys, the logs, and linking 15 seconds to the session timeout.
Insist on the Highest Standards	Q2 unsatisfied with how things were; Q3 improved something “working”	Everyone accepted the blip because nothing was lost. You didn’t.
Learn and Be Curious	Q1 deeper expertise; Q3 didn’t know how to solve it	Learning Kafka rebalancing internals in depth.
Invent and Simplify	Q1 complex problem, simple solution	A system-wide freeze solved mostly through config and clean shutdown.
Ownership	Q1 outside responsibility; Q3 benefits everyone	**[VERIFY]** If deployment infrastructure wasn’t strictly yours.
Earn Trust	Q6 uncovered a significant problem	Explaining an “invisible” problem to the team using dashboard data.
Customer Obsession	Q5 evaluated customer experience	**[VERIFY]** Translate the 15 seconds into delayed rewards or notifications.
Success and Scale	Q8 left it better than you found it	Shared config / runbook protecting every future service.

Follow-up questions to prepare for

- Why exactly 15 seconds? (Know your actual session timeout value.)
- What’s the difference between eager and cooperative rebalancing?
- Why weren’t messages lost? (Committed offsets — consumers resumed where they stopped.)
- How did you verify the fix? (Lag and throughput during a deployment, before vs after.)

Master Story 4

Referral Incentive Product

Slice (Software Developer)

Primary principle: Ownership

Situation

At Slice, growth was a top company priority, and **[VERIFY]** [there was no referral program / the existing one was manual or limited]. Product wanted a referral system: a user shares a code, a friend signs up and completes a qualifying action **[VERIFY]** [e.g. KYC plus a first transaction], and both get a cash reward. I was asked to build it from scratch on the backend.

Task

It paid out real money, so it had to be correct under every condition: no reward paid twice, none missed, and no easy way to game it. **[VERIFY]** It also had to be ready for a [marketing launch] on a fixed timeline.

Action — modelling the flow

[VERIFY] I designed each referral as a state machine: code shared → friend signed up → qualifying action done → reward approved → reward paid. Each step could only move forward from the right previous state, which made the system easy to reason about, audit and debug when users asked "where's my reward?"

Action — concurrent requests

[VERIFY] The same qualifying event could arrive more than once — app retries, duplicate upstream events, or two requests racing at the same instant. I handled it at several levels:

- a database unique constraint so a friend could only be linked to one referral and one reward;
- row-level / optimistic locking on the referral record during state changes, so two simultaneous requests couldn't both move it to "paid";
- a Redis lock around the payout call for the window where an external payment service was involved.

Action — idempotency

[VERIFY] Every reward credit carried an idempotency key built from the referral ID and reward type. A retried request — from the client, a queue or our own retry logic — returned the original result instead of crediting again. This mattered most when the wallet or payment call timed out and we didn't know whether it had succeeded.

Action — fraud prevention

[VERIFY] Once money is attached to sign-ups, people try to farm it. I built checks for the common patterns:

- self-referrals and duplicate identities: matching device IDs, phone numbers and [PAN / bank accounts];
- velocity limits on how many rewards one referrer could earn in a period;
- payout only after a real qualifying action, not just sign-up;
- a holding period before payout, so suspicious referrals could be reviewed or reversed.

I worked with **[VERIFY]** [product / the risk team] on thresholds, to block abuse without frustrating genuine users. **[VERIFY]** I also added dashboards and alerts on funnel conversion, payout volume and blocked-referral rates.

Result

The referral program drove a 150% increase in user onboarding **[VERIFY]** [over X months / vs previous baseline]. **[VERIFY]** We had [zero] duplicate payouts across [X] referrals, and fraud checks blocked about [X% / Rs X] of abusive referrals.

Reflection

[VERIFY] Building something that moves money taught me to design for the failure cases first: retries, races, timeouts and bad actors. Next time I'd [bring the risk team in at design / make fraud rules configurable from the start].

How to angle this story

Principle	Questions it answers	What to emphasize
Ownership	Q1 something significant; Q4 short-term vs long-term	Built end to end; invested in idempotency and fraud checks though a simpler version would launch faster.
Insist on the Highest Standards	Q1 refused to compromise; Q4 standards vs delivery	No duplicate payouts, even under retries and races.
Deliver Results	Q3 exceeded expectations	Lead with the 150%, then how you got there.
Customer Obsession	Q8 customer vs business needs	Fraud checks protect the business; strict rules hurt genuine users.
Frugality	Q1 saved costs	Money saved by blocking fraudulent payouts.
Dive Deep	Q2 root cause	**[VERIFY]** If you investigated a fraud pattern or duplicate-payout bug.
Have Backbone	Q1 disagreed; Q5 pushed back	**[VERIFY]** Only if true: pushing back on launching without fraud checks or idempotency.
Earn Trust	Q5 goals misaligned with another team	**[VERIFY]** If true: growth wanted fewer barriers, risk wanted more.

Follow-up questions to prepare for

- What happens if the payment call times out — did the user get paid or not?
- How did you make sure fraud checks didn't block genuine users?
- 150% compared to what baseline, over what period?
- Why a database constraint and a lock — isn't one enough?

Master Story 5

Replacing Polling With MQTT

Slice (Software Developer)

Primary principle: Invent and Simplify · Frugality

Situation

In the Slice app, a red dot appeared on the profile icon or a navbar icon whenever something needed the user's attention **[VERIFY]** [a new reward, an offer, an unread notification]. To keep it current, the app polled an API every [X seconds] while open. **[VERIFY]** For [X lakh] active users that produced a huge, constant stream of requests, and almost all returned "nothing changed." Load grew with every new user, not with actual activity.

Task

[VERIFY] [I noticed this in endpoint traffic / I was asked to reduce server load.] Keep the red dot accurate — ideally more real-time than before — while removing the wasted requests.

Action — choosing the approach

[VERIFY] I compared:

- Polling less often: easy, but the badge would appear late.
- Firebase push (FCM): good for alerts, but not reliable or fast enough for silent in-app state, and subject to throttling.
- WebSockets: real-time, but we'd build our own pub/sub, reconnection and delivery logic.
- MQTT: lightweight pub/sub built for unreliable mobile networks, with reconnection, delivery guarantees and tiny messages.

I chose MQTT: real-time updates without building the messaging layer ourselves.

Action — designing it

[VERIFY]

- Each app subscribed to its own topic, e.g. user/{id}/badges, on foreground, and disconnected on background to save battery.
- The backend published only when badge state actually changed, triggered by the events that created the reward, offer or notification.
- Retained messages meant a reconnecting app immediately got the latest badge state, with no extra API call.
- Security: token-authenticated connections and topic permissions so users could only subscribe to their own topic.
- Fallback: one sync API call on launch, and slower polling if MQTT couldn't connect.

Action — rolling it out safely

[VERIFY] Released behind a feature flag to [X%] of users first, comparing API traffic, badge accuracy and connection stability before full rollout.

Result

API calls to the badge endpoint dropped by 90%, and server load fell **[VERIFY]** [by X% CPU / X fewer instances]. The red dot became near real-time instead of waiting up to [X seconds] for the next poll. **[VERIFY]** The MQTT setup was later [reused for other real-time features].

Reflection

Polling “worked,” which is why nobody had questioned it. The lesson: look at what the traffic is actually doing, not just whether it succeeds — most requests were asking a question whose answer was almost always “no.”

How to angle this story

Principle	Questions it answers	What to emphasize
Invent and Simplify	Q5 usual approach wouldn’t work; Q1 complex problem, simple solution	Flipping “app keeps asking” to “server tells the app.”
Frugality	Q1 saved costs; Q5 no extra resources	90% fewer API calls and lower server load.
Think Big	Q6 improved a process that was working	Rethought polling instead of tuning it.
Insist on the Highest Standards	Q2 unsatisfied with how things were	Didn’t accept wasteful traffic just because it worked.
Learn and Be Curious	Q7 external trends; Q2 outside comfort area	**[VERIFY]** MQTT comes from IoT — a borrowed idea applied to fintech UI state.
Are Right, A Lot	Q2 alternatives and tradeoffs	Polling vs FCM vs WebSockets vs MQTT.
Customer Obsession	Q3 improved before users asked	Faster badges, less battery and data use.
Earn Trust	Q3 influenced a peer	**[VERIFY]** Only if true: convincing the mobile team or lead to adopt MQTT.

Follow-up questions to prepare for

- Why MQTT and not WebSockets or push notifications?
- What if the connection drops, or the user is offline when the badge changes?
- How many concurrent connections, and which broker?
- How did you measure the 90%?

Master Story 6

Experimentation Platform

Slice (Software Developer)

Primary principle: Think Big · Success and Scale Bring Broad Responsibility

Situation

At Slice we shipped fast, but **[VERIFY]** [a feature went live for everyone as soon as the release went out]. Two problems followed. We couldn't test ideas on a small group first. And some features couldn't legally go live yet: Slice was applying for a PPI (prepaid payment instrument) wallet licence from the RBI, and the wallet had to be fully built in advance but hidden from every user until the licence was granted. We were also building a teen wallet, aimed at **[VERIFY]** [parents, typically over 30, managing wallets for their teenage children], which only made sense for that group.

Task

[VERIFY] [I was asked to / I proposed to] build an experimentation platform: feature flags to control visibility, staggered rollouts to release gradually, and cohort targeting for specific user groups, with tracking for proper A/B tests.

Action — the flag system

[VERIFY]

- Flags evaluated on the server, so nobody could enable a hidden feature by modifying the app — critical for a regulated product.
- Rules targeting users by attributes like age, city, platform and app version, enabling cohort rollouts.
- Consistent hashing on user ID for percentage rollouts, so a user stayed in the same group across sessions.
- Flag configs cached in Redis, adding almost no latency.
- Every flag change logged with who and when — an audit trail for regulated features.
- Every flag doubled as a kill switch, turning a feature off instantly without a release.

Action — staggered rollouts and A/B tests

[VERIFY] Features rolled out in stages (1% → 5% → 25% → 100%) with error rates, latency and business metrics checked at each step. The platform recorded which variant each user saw, so product teams decided with data rather than opinion.

Action — PPI wallet and teen wallet

[VERIFY] The wallet stayed behind a flag, off for all real users but available to internal testers, for about a year while the licence was pending. When the licence came through, launch was a configuration change, rolled out in stages over [X days]. The teen wallet went only to the target cohort [parents over 30], starting small and expanding as engagement was confirmed.

Result

[VERIFY] The platform launched [X] features, including the PPI wallet and teen wallet. After the licence was granted, the wallet went live within [X days] with no last-minute code changes. Staggered rollouts caught [X issues] at the 1–5% stage. Product teams moved to [X A/B tests per month / data-backed launches].

Reflection

[VERIFY] Building a shared platform, rather than one-off switches, made it valuable. It turned “is it safe to launch?” from an all-or-nothing decision into a gradual one we could reverse at any point.

How to angle this story

Principle	Questions it answers	What to emphasize
Think Big	Q1 bigger opportunity than initial focus	**[VERIFY]** If true: the need was a few flags; you built a company-wide platform.
Ownership	Q4 long-term over short-term	Wallet built a year ahead of the licence; infrastructure that kept paying off.
Success and Scale	Q1 impact beyond customer; Q4 responsibility in decisions	An unlicensed financial product could never reach users; audit trail.
Customer Obsession	Q9 customer base not yet served (DEI)	Teen wallet served families; cohort rollout to learn their needs.
Invent and Simplify	Q6 novel idea, big impact; Q3 made things simpler	Launch became a config change, not a release.
Bias for Action	Q1 calculated risk	Kill switches and staged rollouts made launches reversible.
Are Right, A Lot	Q3 many sources of input	A/B testing replaced opinion with data.
Insist on the Highest Standards	Q6 continuous improvement	Staged rollouts caught problems before most users saw them.

Follow-up questions to prepare for

- How did you ensure a user always saw the same variant?
- What if the flag service went down? (Cached configs / safe default of “off”.)
- How did you decide whether an A/B result was meaningful?
- How did you verify age for the teen wallet cohort? **[VERIFY]** [e.g. KYC data]

Master Story 7

Accessibility Fix for a Blind Customer

Slice (Software Developer)

Primary principle: Insist on the Highest Standards · Customer Obsession · Deliver Results

Situation

We received a support ticket from a blind customer who couldn't repay his loan in the Slice app. He used his Android phone's screen reader **[VERIFY]** [TalkBack / read-aloud], but on our repayment screens it wasn't reading the buttons and fields properly, so he couldn't complete the payment. He wanted to pay and couldn't, which could have meant **[VERIFY]** [late fees and a credit-score impact] through no fault of his own.

Task

[VERIFY] [I picked up the ticket.] Find out why the screen reader wasn't working, fix it quickly so he could repay, and make sure it didn't affect other customers.

Action — reproducing the problem

[VERIFY] I turned on TalkBack and went through the repayment flow without looking at the screen. Many elements were skipped entirely or announced only as "button" or "unlabelled." The pay button, amount options and several icons gave no useful information. For a sighted user the screen looked perfect; for him it was unusable.

Action — root cause

[VERIFY] We had recently migrated from XML layouts to Jetpack Compose. In XML, labels came from android:contentDescription, and standard Buttons and TextViews were read automatically. In Compose, screen readers rely on the semantics tree, which developers must populate explicitly. During the migration:

- Icons and images were written with contentDescription = null, so TalkBack ignored them, including icon-only buttons.
- Custom components built from a Box or Row with Modifier.clickable declared no role (e.g. Role.Button) or click label.
- Cards made of several small elements were read as fragments, because they weren't grouped with Modifier.semantics(mergeDescendants = true).

I confirmed this with Android's Accessibility Scanner and the Layout Inspector.

Action — fixing it for him first

[VERIFY] I prioritised the repayment flow: content descriptions, roles and click labels on every element, retested with TalkBack until the whole flow worked without looking at the screen, and shipped it as [a hotfix / next release]. Support confirmed he was able to repay. [We also made sure he wasn't charged late fees.]

Action — across the app, and preventing recurrence

[VERIFY] Since the same gap existed everywhere, I added missing semantics to almost every component: icon descriptions, roles for clickables, grouping for cards, and headings for section titles. Then:

- made contentDescription a required parameter in our shared Compose components;
- added UI tests that find and use key screens through accessibility labels;
- added an accessibility check (TalkBack / Accessibility Scanner) to the [PR checklist / QA process].

Result

[VERIFY] The customer repaid his loan. I fixed accessibility labels on about [X] screens / [X] components, and Accessibility Scanner issues dropped from [X] to [Y]. New screens were accessible by default.

Reflection

The bug was invisible to everyone who tested visually, which is why it slipped through. Accessibility can’t be an afterthought or a one-time fix; it has to live in the components and the review process, because the people affected often have no other way to use the product.

How to angle this story

Principle	Questions it answers	What to emphasize
Insist on the Highest Standards	Q7 accessibility issue (DEI)	Near-exact match. Customer first, then the systematic fix.
Deliver Results	Q7 customers with diverse needs (DEI)	Near-exact match. How you audited the experience (TalkBack, Scanner).
Customer Obsession	Q1 difficult customer situation; Q2 above and beyond	One ticket led to fixing the whole app.
Think Big	Q7 accessibility for all (DEI)	From one-off fix to accessible-by-default.
Dive Deep	Q2 root cause	Tracing it to the XML-to-Compose migration.
Ownership	Q1 beyond your responsibility	**[VERIFY]** You could have fixed only his screen.
Success and Scale	Q5 everyone affected by your work	One case revealed a hidden group of excluded users.
Bias for Action	Q4 respond immediately	A customer unable to repay a loan is urgent.

Follow-up questions to prepare for

- How is accessibility handled differently in Compose vs XML?
- How many users do you think were affected?
- How did you make sure the fix lasted?
- What did you do for that specific customer?

Master Story 8

The Stale Cache I Shipped

Slice (Software Developer)

Primary principle: Earn Trust

Template — make it real: Was there a time your caching or another change caused a problem? Use that real bug. If not, use any real mistake you owned openly — the structure stays the same.

Situation

At Slice, I added Redis caching to key services to reduce database load and p99 latency. **[VERIFY]** One was the [rewards / user profile] service, handling [X requests per second] and putting heavy load on PostgreSQL.

Task

Make the service faster without changing what users saw.

Action

I added a cache in front of the main reads with a [X-minute] expiry. p99 latency dropped from [X ms] to [Y ms] and database load fell by [X%]. **[VERIFY]** A few days later, support flagged users seeing outdated data — [a reward just earned didn't appear] for several minutes.

I traced it to my change: I had cached data that another service could update, relying only on expiry rather than clearing the cache when the data changed. I told my lead and team directly — what went wrong, who was affected, and how I'd fix it — instead of quietly patching it. Then I fixed it properly:

- invalidated the cached entry whenever the underlying data changed;
- read the most sensitive fields from the database instead of the cache;
- added a metric comparing cache and database values on a sample of requests, so staleness would appear on a dashboard, not in a support ticket.

[VERIFY] I wrote a short [postmortem / team note] with a checklist for anyone adding caching: who else writes this data, and how will the cache find out?

Result

The stale-data issue stopped while the latency gain stayed (p99 around [Y ms]). **[VERIFY]** The checklist was [used on later caching changes].

Reflection

The fix was technical, but being open about it mattered most. Owning the mistake quickly meant the team trusted the fix — and trusted me with the next performance project.

How to angle this story

Principle	Questions it answers	What to emphasize
Earn Trust	Q4 couldn't meet a commitment; Q6 uncovered a significant problem	Vocally self-critical: owning the mistake openly.
Are Right, A Lot	Q4 bad decision; Q5 error in judgment	The cache design flaw and what you changed.
Insist on the Highest Standards	Q5 goal you wish you'd done better	Latency win came with a correctness gap.

Follow-up questions to prepare for

- Why didn’t you invalidate on write from the start?
- How did you find which users were affected?
- What’s your rule now for deciding what to cache?

Master Story 9

Unified Entitlements

StonksAI (Co-Founder)

Primary principle: Have Backbone; Disagree and Commit

Template — make it real: Use a real disagreement with your co-founder or a Slice colleague (architecture, scope, timelines). Also prepare one real example where you LOST the argument and still committed fully — Disagree and Commit Q3/Q4 need that.

Situation

At StonksAI we were launching paid subscriptions on two platforms: Razorpay on Android and RevenueCat on iOS, each sending its own webhooks in its own format.

Task

Decide how the backend would track whether a user had an active subscription.

Action

[VERIFY] My co-founder wanted to handle each platform separately and ship faster — a reasonable view, since speed mattered. I disagreed: with two sources of truth, a user who pays on one platform and logs in on another, or whose renewal webhook arrives late, can end up paying but locked out — the fastest way to lose trust and get refund requests.

Instead of just arguing, I made the case concretely:

- listed failure scenarios: cross-device login, late or duplicate webhooks, refunds, expiry and renewal;
- estimated the cost: a unified service would take about [X extra days], while separate logic meant every new feature checking two systems forever;
- proposed a time-box: [X days] for the unified service, and if it overran we'd ship separately and migrate later.

[VERIFY] My co-founder agreed. I built one service that received webhooks from both platforms, validated receipts, and produced a single subscription state per user that the whole app read from.

Result

We shipped within the time-box. **[VERIFY]** In the first [X weeks] we saw [X] cross-platform users and [X] late webhooks that would have locked out paying users under the separate approach. New paid features needed one check instead of two.

Reflection

Disagreeing with data and a fallback plan made it a joint decision rather than a fight. **[VERIFY]** Had my co-founder chosen the separate approach anyway, I'd have committed fully and planned the migration for later.

How to angle this story

Principle	Questions it answers	What to emphasize
Have Backbone; Disagree and Commit	Q1 disagreed on something important; Q2 unpopular stance	Respectful challenge backed by data and a fallback.
Earn Trust	Q3 influenced a peer with a different opinion	Making the case concretely rather than by opinion.
Ownership	Q4 long-term over short-term	A few extra days now vs two systems forever.

Principle	Questions it answers	What to emphasize
Customer Obsession	Q8 customer vs business needs	Paying users must never be locked out.

Follow-up questions to prepare for

- What would you have done if the time-box ran out?
- How did you handle webhooks arriving out of order?
- Tell me about a time you disagreed and lost.

Master Story 10

Making On-Call Bearable

Slice (Software Developer)

Primary principle: Strive to Be Earth's Best Employer

Template — make it real: Use a real time you helped a struggling teammate, mentored a junior or intern, or improved how your team worked. Keep the SLO dashboard work (on your CV) as the lasting-fix part.

Situation

[VERIFY] At Slice, on-call was stressful. Most incidents were reported by customers or support before our team noticed, alerts were missing or noisy, and there were no clear steps to follow. **[VERIFY]** One night a deployment by [a junior engineer] caused errors on [a payments-related service]. They were on call, didn't know where to look, and were clearly panicking.

Task

Help restore the service, support my teammate, and fix the reasons on-call was so painful for everyone.

Action — supporting a teammate under pressure

[VERIFY] I joined the call and kept things calm. I didn't take over; I walked them through it — checking recent deployments, reading error logs, deciding to roll back. They did the rollback themselves and service recovered in [X minutes]. I made sure the review was blameless — the cause was missing safeguards, not one person — and credited them for the fix in the team review.

Action — fixing the underlying problem

That incident showed on-call depended on knowing where to look. So I:

- set up SLO dashboards on Grafana for critical services — latency, error rates, uptime — so health was visible at a glance;
- configured breach alerts on those SLOs and **[VERIFY]** removed or retuned noisy alerts that fired without needing action;
- **[VERIFY]** wrote short runbooks for the most common alerts: what it means, where to look, how to roll back;
- **[VERIFY]** paired with [the junior engineer] on building some dashboards, so they learned the system and could own it.

Result

[VERIFY] We started catching incidents before customers reported them ([X%] detected by alerts, up from [Y%]). Time to resolve dropped from about [X] to [Y] minutes and noisy alerts fell by [X%]. Newer engineers handled on-call confidently, and [the junior engineer later led on-call for the service].

Reflection

The tooling helped, but the bigger change was how people felt on-call. When people know they won't be blamed and have clear steps, they own problems instead of fearing them.

How to angle this story

Principle	Questions it answers	What to emphasize
Strive to Be Earth's Best Employer	Q3 improved coworkers' experience; Q5 compassion under pressure; Q6 supported someone's goal; Q2 advocated for someone less senior	Blameless culture, credit-sharing, and better on-call tooling.
Earn Trust	Q8 helped a struggling team member	Supporting without taking over.
Ownership	Q6 saw a peer struggling and helped	Stepping in, then fixing the system.
Hire and Develop the Best	Q4 coached a team member (non-manager)	Pairing with the junior engineer on dashboards.
Bias for Action	Q6 proactively fixed a team issue	Turning one incident into lasting tooling.

Follow-up questions to prepare for

- How did you decide which alerts were noise?
- How did you pick the SLO targets?
- How did your teammate respond afterwards?

Final prep notes

Fill every bracket before the interview. Interviewers probe Results hardest. Real, specific numbers (latency, %, users, Rs) are far more convincing than round estimates.

Verify or replace every [VERIFY] detail. Anything you cannot explain under three follow-up questions will hurt you more than leaving it out.

Say “I”, not “we”. Amazon assesses your contribution. Describe the team context briefly, then what you personally decided and built.

Prepare a real “disagree and lost” story. Disagree and Commit questions need a case where you did not get your way and still committed fully.

NPS platform (3.3 → 4.5 Play rating). Expect “how did NPS raise the rating?” Explain it as using feedback to find and fix the main complaints. Be careful if the answer involves showing the Play Store review prompt only to happy users — Google’s guidelines discourage asking users their opinion right before the review prompt.

Keep each answer to about two minutes. Roughly: Situation and Task 20%, Action 60%, Result and Reflection 20%. Let the interviewer pull more detail through follow-ups.